

AWS CLOUD + DEVOPS

PORTFOLIO ROADMAP

Ten progressive, production-style projects
from first pipeline to enterprise platform

MISSION

Build evidence that you can design, deploy, observe, secure,
recover and operate AWS platforms - not only create resources.

Prepared for Soufiane

Roadmap version: 5 July 2026

Contents

The project numbers preserve the original mission. The recommended execution sequence introduces baseline monitoring before progressive deployment and evolves one platform rather than creating disconnected demos.

How to use this roadmap	2
Portfolio definition of done	4
01. Complete CI/CD with Jenkins, ECR, EC2 and Fargate	5
02. Production Infrastructure with Terraform	7
03. Enterprise Blue/Green and Canary Deployment	9
04. Monitoring and Observability Platform	11
05. Automated Disaster Recovery and Recovery Testing	13
06. Cloud Security Detection and Safe Remediation	15
07. Event-Driven Serverless Automation	17
08. Kubernetes Application Platform on Amazon EKS	19
09. GitOps and Infrastructure Delivery	21
10. Complete Enterprise Cloud Platform	23
Mission playbook	25
Reference baseline	27

How to use this roadmap

CORE STRATEGY

Use one professional GitHub account, one repository per project and one portfolio index. Reuse WellSpring as the workload so each project demonstrates a new operational capability rather than another unrelated application.

Recommended execution sequence

- Project 1: direct EC2 deployment first, then an ECS Fargate rolling deployment.
- Project 2: codify the shared production foundation with Terraform.
- Project 4 baseline: create essential logs and alarms before automating progressive traffic shifting.
- Project 3: implement ECS-native blue/green or canary deployment with alarm rollback.
- Project 7: add event-driven, serverless engineering experience.
- Project 5: introduce a small stateful backend, then design and test disaster recovery.
- Project 6: centralize security detection and safe remediation.
- Project 8: build the EKS platform after demonstrating strong ECS fundamentals.
- Project 9: combine Terraform delivery with true pull-based Kubernetes GitOps.
- Project 10: integrate the proven components into the capstone platform.

Modern design decisions

ECS DEPLOYMENTS IN 2026

Prefer ECS-native blue/green, canary or linear deployment for a new design. CodeDeploy remains a useful optional comparison lab, but it is no longer the default recommendation for greenfield ECS progressive delivery.

GITOPS TERMINOLOGY

GitHub Actions running Terraform is infrastructure CI/CD. True GitOps adds a pull-based reconciler such as Argo CD or Flux that continuously aligns the Kubernetes cluster with Git.

Portfolio definition of done

A project is complete only when another engineer can understand it, reproduce it, observe it failing and follow the documented recovery path.

Required in every repository

- README with problem statement, architecture, prerequisites, setup, verification and teardown.
- Architecture diagram showing system, network or trust boundaries and important traffic flows.
- Infrastructure as code with format, validation, lint and security checks.
- Pipeline as code and immutable artifact or release versioning.
- Least-privilege IAM, temporary credentials and explicit secret handling.
- Health checks, useful logs, metrics, alarms and a failure scenario.
- Rollback or recovery runbook that has been executed at least once.
- Cost considerations, tags and a reliable destroy or cleanup procedure.
- Sanitized screenshots, logs or a short demo proving the result.
- Lessons learned and the next improvement you would make in a real production environment.

Repository naming

Profile	A public repository matching your GitHub username, containing a concise profile README.
Portfolio index	cloud-devops-portfolio - links all projects and explains the progression.
Project repos	01-jenkins-ec2-ecs-cicd through 10-enterprise-cloud-platform.
Sensitive files	Never commit .pem keys, AWS credentials, Terraform state, secrets, production logs or real personal data.

Complete CI/CD with Jenkins, ECR, EC2 and Fargate

Build a complete delivery pipeline for WellSpring. Begin with Docker running directly on one EC2 target, then evolve the same image and pipeline into an ECS Fargate rolling deployment.

ARCHITECTURE FLOW

GitHub -> Jenkins on EC2 -> Docker build -> Amazon ECR -> SSM -> Target EC2 / Fargate

Primary outcome	A source commit produces a tested, uniquely tagged container image and deploys it automatically with a verified health check.
Prerequisites	Git, GitHub, Docker basics, Linux commands, IAM fundamentals and one AWS account with cost controls.
Key services	EC2, ECR, IAM, Systems Manager, ECS, Fargate, CloudWatch Logs
Repository	01-jenkins-ec2-ecs-cicd

Implementation phases

Phase A - Container baseline

Validate the existing Dockerfile, build locally, run the Nginx health endpoint and document the image contract.

Phase B - Jenkins pipeline

Create checkout, validate, build, scan, tag and push stages. Use an EC2 instance role instead of stored AWS access keys.

Phase C - Direct EC2 deployment

Use Systems Manager Run Command to pull the exact ECR tag, replace the running container and verify /health.

Phase D - ECS Fargate

Create an ECS cluster, task definition and service. Change deployment from docker run to a new task-definition revision and rolling service update.

Phase E - Rollback

Retain the previous known-good image or task revision and automatically restore it when the smoke test fails.

Required deliverables

- Jenkinsfile stored in Git, Dockerfile, Nginx health endpoint and image-tagging convention.
- Separate Jenkins and application EC2 IAM roles with least-privilege permissions.
- ECR repository with image scanning and lifecycle cleanup.
- Direct EC2 deployment script plus an ECS Fargate task definition and service update flow.
- Architecture diagram, setup guide, rollback runbook and pipeline screenshots.

Definition of success

- A commit to the deployment branch starts the pipeline without manual server login.
- Every build uses an immutable tag containing build number and commit SHA.
- A failed validation or image scan stops before deployment.

PROJECT 01 / CI/CD

- The target health endpoint returns HTTP 200 after deployment.
- The previous release can be restored and the action is visible in Jenkins logs.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Publish an intentionally broken Nginx configuration or health endpoint. Confirm that the pipeline detects the failure, reports useful logs and restores the previous version.

Portfolio evidence

- Jenkins stage view and console output with sensitive values hidden.
- ECR repository showing immutable commit-based tags.
- Before-and-after application version, SSM command result and ECS service revision.
- Short incident note describing the rollback test and recovery time.

Security and cost guardrails

- Never expose Jenkins port 8080 to the whole internet; use a restricted security group and later HTTPS.
- Do not keep SSH private keys or permanent AWS keys in Jenkins.
- Use separate Jenkins and workload roles; Docker access on Jenkins is effectively privileged.
- Stop or destroy practice EC2 and Fargate resources when not in use.

Production Infrastructure with Terraform

Replace manually created infrastructure with repeatable, reviewable Terraform. Build a reusable AWS foundation for later ECS, monitoring, security and disaster-recovery projects.

ARCHITECTURE FLOW

GitHub PR -> Terraform checks -> S3 remote state -> AWS provider -> VPC + ALB + ECS

Primary outcome	A clean AWS environment can be created, changed and destroyed predictably from version-controlled Terraform.
Prerequisites	Project 1, Terraform CLI, AWS networking basics and an understanding of state and IAM roles.
Key services	VPC, ALB, ECS, ECR, IAM, S3, CloudTrail
Repository	02-terraform-production-infrastructure

Implementation phases

Bootstrap

Create a dedicated encrypted and versioned S3 state bucket. Enable native S3 locking with `use_lockfile=true` and restrict access to pipeline and break-glass roles.

Network modules

Create a multi-AZ VPC, public and private subnets, routes, security groups and carefully chosen egress connectivity.

Workload modules

Create ECR, ALB, target groups, ECS cluster, task definition, service, IAM task roles and log groups.

Environment isolation

Use separate state and configurations for development, staging and production. Avoid relying on workspaces as the only production boundary.

Quality gates

Run `fmt`, `validate`, `TFLint`, security scanning and plan in pull requests. Require approval before production apply.

Required deliverables

- Bootstrap configuration for the state bucket and access policy.
- Reusable modules for network, security, ECR, ALB and ECS.
- Environment-specific root configurations with variables and outputs.
- Automated `fmt`, `validate`, `lint`, security and plan checks.
- Dependency diagram, module documentation, cost notes and teardown guide.

Definition of success

- A new environment can be created from an empty account without console-created dependencies.
- A second concurrent apply is blocked by state locking.

PROJECT 02 / TERRAFORM

- Plans contain no unexpected replacement or destructive operations.
- terraform destroy removes project resources without touching shared state infrastructure.
- No credentials, secrets or state files exist in Git history.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Simulate a failed apply and state-lock contention. Recover safely without manually editing the state file, then document the recovery procedure.

Portfolio evidence

- Module tree and generated plan summary from a pull request.
- AWS resource view showing consistent Project, Environment, Owner and ManagedBy tags.
- State bucket versioning, encryption and lockfile configuration.
- A successful create-change-destroy lifecycle log.

Security and cost guardrails

- Never commit terraform.tfstate, .terraform directories, secret variable files or plan files containing secrets.
- Pin Terraform and provider versions and commit the dependency lock file.
- Use least-privilege pipeline roles and short-lived credentials.
- NAT Gateways, ALBs and continuously running tasks incur cost; document and automate teardown.

Enterprise Blue/Green and Canary Deployment

Deploy a new ECS service revision beside the stable revision, validate it, shift production traffic safely and roll back automatically when health or business alarms fail.

ARCHITECTURE FLOW

Pipeline -> ECS service revision -> Green tasks -> ALB weighted target groups -> CloudWatch alarms -> Rollback

Primary outcome	Near-zero-downtime releases with controlled traffic shifting, validation hooks, bake time and automatic rollback.
Prerequisites	Projects 1 and 2, plus baseline alarms from Project 4 implemented before production-style traffic shifting.
Key services	ECS Fargate, ALB, CloudWatch, Lambda hooks, SNS, ECR
Repository	03-ecs-progressive-deployment

Implementation phases

ECS-native baseline

Use the ECS deployment controller and BLUE_GREEN strategy. ECS-native blue/green, canary and linear strategies are the preferred greenfield path in 2026.

Traffic and health

Configure production and optional test listeners, target groups, container health checks, ALB health checks and a deployment bake period.

Validation hooks

Run automated smoke tests against green before production traffic, then validate again during and after traffic shifting.

Alarm rollback

Attach latency, 5xx and healthy-target alarms. A breached alarm must pause or fail the deployment and return traffic to blue.

Optional legacy comparison

Implement CodeDeploy once only as a comparison lab if desired; document why ECS-native is simpler for a new 2026 design.

Required deliverables

- Terraform for ALB listeners, target groups, ECS deployment strategy, alarms and IAM roles.
- Green smoke-test hook and deployment status notification.
- Blue/green, canary or linear traffic-shift configuration with a documented bake time.
- Automatic rollback demonstration and operational runbook.
- Sequence diagram showing preparation, test, shift, bake and cleanup phases.

Definition of success

- Blue remains available while green starts and validates.
- Production traffic moves according to the selected strategy without visible downtime.
- A breached alarm automatically returns traffic to the stable revision.
- The failed revision and alarm reason are visible in deployment events.
- Old tasks are removed only after the configured bake time.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Deploy a version that returns HTTP 500 on a controlled endpoint. Send canary traffic, trigger the 5xx alarm and capture proof that ECS rolls back to blue.

Portfolio evidence

- ALB target-group and ECS deployment timelines.
- CloudWatch alarm transition and rollback event.
- Traffic-shift visualization plus successful requests during the deployment.
- Comparison note: ECS-native versus CodeDeploy responsibilities.

Security and cost guardrails

- Blue and green run together temporarily, so deployments can roughly double compute usage during bake time.
- Use alarms based on customer symptoms, not only CPU.
- Do not terminate blue immediately; preserve a deliberate rollback window.
- Keep lifecycle hooks idempotent and time-bounded.

Monitoring and Observability Platform

Create useful operational visibility across EC2, ECS and the application using structured logs, metrics, alarms, dashboards, notifications and documented response procedures.

ARCHITECTURE FLOW

Application + infrastructure -> Logs / metrics / traces -> CloudWatch -> Alarms + dashboard -> SNS notification -> Runbook

Primary outcome	An operator can detect, understand and respond to a failure using dashboards and alerts rather than logging into servers blindly.
Prerequisites	Project 1 workload, basic CloudWatch understanding and an email endpoint for controlled SNS testing.
Key services	CloudWatch Logs, CloudWatch Metrics, CloudWatch Agent, Container Insights, SNS, X-Ray / OpenTelemetry
Repository	04-aws-observability-platform

Implementation phases

Signal design

Define user-facing indicators: availability, latency, error rate and request volume. Establish initial service-level objectives and alert thresholds.

Logs

Send structured application and access logs to CloudWatch with retention, correlation IDs and filters for errors. Never log secrets or health data.

Metrics

Collect EC2 memory and disk metrics with CloudWatch Agent, ECS service metrics, ALB metrics and selected custom business metrics.

Dashboards and alarms

Build one operational dashboard and actionable alarms for ALB 5xx, latency, healthy targets, ECS task health and deployment failure.

Tracing and runbooks

Add OpenTelemetry or X-Ray when a backend exists. Link each alarm to a concise diagnosis and recovery runbook.

Required deliverables

- CloudWatch Agent configuration for EC2 and log configuration for ECS tasks.
- CloudWatch dashboard provisioned through Terraform.
- SNS topic, subscriptions and alarms with severity and ownership labels.
- Log retention policy, metric filters and optional distributed tracing.
- SLO document, alarm runbooks and an observability architecture diagram.

Definition of success

- A real failure triggers the intended alarm and notification within the documented detection time.
- The dashboard shows traffic, errors, latency and resource health together.

PROJECT 04 / OBSERVABILITY

- Logs identify the affected release and request without revealing sensitive data.
- An operator can follow the linked runbook to identify and mitigate the issue.
- Log retention and high-cardinality metric costs are controlled.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Stop a task or generate controlled HTTP 500 responses. Measure detection time, notification time and recovery time, then improve any noisy or missing signals.

Portfolio evidence

- Dashboard screenshot during normal operation and during the incident.
- Alarm history, SNS delivery and related log query.
- A short incident timeline with detection and recovery measurements.
- Runbook excerpt showing the exact decision path.

Security and cost guardrails

- Alert on symptoms users feel; avoid creating alarms for every available metric.
- Set log retention explicitly and avoid unbounded debug logging.
- Redact tokens, credentials and personal or health-related information.
- CloudWatch custom metrics, high-volume logs and tracing can become expensive; sample and retain deliberately.

Automated Disaster Recovery and Recovery Testing

Design recovery around business targets, automate backups and replication, then prove recoverability through scheduled restore tests and documented game days.

ARCHITECTURE FLOW

Primary region -> AWS Backup + snapshots -> Cross-region copies -> Recovery Terraform -> Restore test
-> DNS cutover

Primary outcome	The platform can recover data and service in a measured time that satisfies documented RPO and RTO objectives.
Prerequisites	Terraform foundation, a small stateful backend with synthetic data, monitoring and a clear understanding of application dependencies.
Key services	AWS Backup, EBS, RDS, S3 Replication, KMS, Route 53, EventBridge
Repository	05-aws-disaster-recovery

Implementation phases

Business targets

Define recovery point objective, recovery time objective, dependency map and backup ownership before choosing technology.

Protection

Configure EBS snapshots, RDS automated backups and point-in-time recovery, S3 versioning and cross-region replication, plus cross-region KMS permissions.

Recovery environment

Use Terraform provider aliases and separate state to create the minimum viable recovery infrastructure in a second region.

Automated verification

Schedule restore jobs into isolated infrastructure, validate data and application health, capture results and destroy temporary resources.

Game day

Simulate regional or data-loss scenarios, execute the runbook, measure actual RPO/RTO and record improvements.

Required deliverables

- Recovery strategy document with RPO, RTO, scope and assumptions.
- AWS Backup vault, plans, retention, encryption and cross-region copy rules.
- RDS restore and S3 replication validation automation.
- Terraform recovery-region configuration and optional Route 53 failover record.
- Game-day report with actual recovery measurements and corrective actions.

Definition of success

- Backups complete and failed backup jobs generate alerts.

PROJECT 05 / DISASTER RECOVERY

- A randomly selected recovery point can be restored and validated automatically.
- Cross-region replicated objects remain encrypted and readable by the intended role.
- Measured recovery meets the declared RPO and RTO or produces a concrete improvement plan.
- Recovery tests cannot overwrite production data.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Delete or corrupt synthetic application data, restore from a selected recovery point and verify integrity.
Later simulate loss of the primary region without changing real production DNS.

Portfolio evidence

- Backup job history, restore job and validation output.
- Primary and recovery-region architecture diagram.
- RPO/RTO scorecard with measured rather than assumed values.
- Game-day timeline, decisions, gaps and follow-up actions.

Security and cost guardrails

- Backups are not disaster recovery until restore is tested.
- Use synthetic data; do not place real medical or personal information in the portfolio environment.
- Protect backup vaults from the same credentials used for ordinary workload administration.
- Cross-region storage, standby resources and data transfer add ongoing cost; start with backup-and-restore.

Cloud Security Detection and Safe Remediation

Establish preventive, detective and responsive controls using AWS security services, infrastructure-as-code scanning and carefully constrained automated remediation.

ARCHITECTURE FLOW

AWS resources -> Config + Security Hub + GuardDuty -> EventBridge -> Lambda remediation -> SNS + audit logs

Primary outcome	Misconfigurations and threats are detected centrally, prioritized and remediated safely with a complete audit trail.
Prerequisites	Terraform, IAM, CloudTrail, EventBridge and Lambda fundamentals plus a separate non-production testing environment.
Key services	AWS Config, Security Hub, GuardDuty, Inspector, IAM Access Analyzer, Lambda, EventBridge, CloudTrail
Repository	06-aws-security-automation

Implementation phases

Foundations

Enable CloudTrail, secure log storage, IAM Access Analyzer and essential account-level security settings. Document ownership and escalation.

Detection

Aggregate findings from Security Hub, GuardDuty, Inspector and Config with severity, resource context and deduplication.

Shift-left controls

Scan containers, dependencies, secrets and Terraform in pull requests. Block critical issues before deployment.

Safe remediation

Start with reversible actions such as adding required tags, disabling public access or quarantining a security group. Require approval for destructive actions.

Validation

Generate controlled findings or non-compliant resources and verify detection, notification, remediation and audit evidence.

Required deliverables

- Security services and Config rules provisioned by Terraform.
- EventBridge routing, SNS notifications and idempotent remediation Lambda functions.
- CI security gates for IaC, images, dependencies and accidental secrets.
- Finding triage matrix with severity, owner, SLA and response action.
- Threat model, remediation safety policy and test report.

Definition of success

- A controlled misconfiguration creates a finding with correct resource context.
- Critical findings notify the correct destination and lower-severity noise is filtered.
- Automatic remediation is reversible, idempotent and limited by tags or account scope.
- Every change records the finding, decision, Lambda execution and resulting resource state.
- The pipeline rejects a deliberately vulnerable container or unsafe Terraform change.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Create a temporary public S3 bucket or overly broad security group in a sandbox. Verify detection and safe quarantine, then test that protected resources cannot be modified by the remediation role.

Portfolio evidence

- Security Hub or Config finding before and after remediation.
- EventBridge event, Lambda log and CloudTrail audit record.
- CI failure caused by an intentionally unsafe IaC change.
- Threat model showing preventive, detective and responsive controls.

Security and cost guardrails

- Do not begin by automatically deleting resources; quarantine or notify first.
- Use a tightly scoped remediation role and explicit resource tags.
- Security Hub, GuardDuty, Inspector and Config generate ongoing charges; scope the lab and disable unused controls afterward.
- Never publish real account IDs, secrets, findings containing sensitive data or unredacted screenshots.

Event-Driven Serverless Automation

Process uploaded objects asynchronously and reliably. Turn an S3 event into validated metadata in DynamoDB, user notification through SNS and searchable operational logs.

ARCHITECTURE FLOW

S3 upload -> EventBridge / S3 event -> Lambda -> DynamoDB -> SNS -> CloudWatch Logs + DLQ

Primary outcome	A resilient, idempotent event-processing workflow that handles retries, duplicates and failures without losing work.
Prerequisites	Python or JavaScript basics, IAM, Lambda, S3 events and DynamoDB fundamentals.
Key services	S3, EventBridge, Lambda, DynamoDB, SNS, SQS DLQ, CloudWatch
Repository	07-event-driven-aws-automation

Implementation phases

Event contract

Define the object naming, metadata, supported content types, event schema and expected processing result.

Processing

Validate the object, calculate metadata or perform a harmless transformation, then write an idempotent result to DynamoDB.

Reliability

Handle duplicate delivery, partial failure, retries and poison events. Add a dead-letter queue or failure destination.

Notification

Publish a success or failure event through SNS without exposing sensitive object content.

Operations

Create logs, metrics, alarms, tracing and a replay procedure for failed events.

Required deliverables

- Terraform for buckets, event routing, functions, table, topic, DLQ and least-privilege roles.
- Versioned Lambda code with unit tests and structured logging.
- Idempotency key and DynamoDB conditional-write strategy.
- Failure alarm, DLQ replay script and operational runbook.
- Event schema and sequence diagram.

Definition of success

- One valid upload creates exactly one logical DynamoDB result even when the event is delivered twice.
- Unsupported files fail safely and produce an observable event.
- Retries do not create duplicate notifications or records.
- A poisoned event reaches the DLQ and can be replayed after correction.
- All resources are deployed and removed through Terraform.

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Upload a malformed file and deliberately fail the Lambda. Observe retries and DLQ delivery, fix the handler and replay the message successfully.

Portfolio evidence

- Sequence diagram and sample sanitized event payload.
- DynamoDB record, SNS notification and correlated CloudWatch log.
- Duplicate-delivery test proving idempotency.
- DLQ replay demonstration and failure alarm.

Security and cost guardrails

- Validate object type and size; never trust uploaded content.
- Restrict Lambda access to the exact bucket prefix, table and topic.
- Avoid recursive S3 triggers by separating input and output prefixes or buckets.
- Set retention and concurrency controls to prevent runaway cost.

Kubernetes Application Platform on Amazon EKS

Deploy WellSpring and a small backend on managed Kubernetes with scalable workloads, ingress, secure workload identity, observability and controlled upgrades.

ARCHITECTURE FLOW

GitHub + ECR -> EKS control plane -> Managed nodes -> Deployment + Service -> ALB Ingress -> Users

Primary outcome	A production-style Kubernetes workload that scales, survives pod failure and exposes traffic through an AWS-managed load balancer.
Prerequisites	Containers, Terraform, networking, IAM, ECS experience and practical Kubernetes fundamentals.
Key services	EKS, EC2 managed nodes, ECR, ALB, CloudWatch, IAM Pod Identity
Repository	08-amazon-eks-platform

Implementation phases

Cluster foundation

Provision VPC, EKS control plane, managed node group, security groups, encryption and access entries with Terraform.

Workload

Create namespace, Deployment, Service, ConfigMap, Secret integration, resource requests, limits and health probes.

Ingress

Install AWS Load Balancer Controller with workload identity and expose the application through an ALB Ingress.

Scaling and resilience

Install metrics support, configure HPA and PodDisruptionBudget, then evaluate Cluster Autoscaler or Karpenter.

Operations

Collect container logs and metrics, scan images, test node and pod failures and document cluster upgrades.

Required deliverables

- Terraform modules for EKS, nodes, IAM and network dependencies.
- Kubernetes manifests or Helm chart for the application.
- AWS Load Balancer Controller, ingress, TLS plan and DNS design.
- HPA, resource policy, disruption budget and workload identity configuration.
- Cluster operations, upgrade, troubleshooting and teardown runbooks.

Definition of success

- The application remains available when a pod is deleted.
- HPA increases replicas during a controlled load test and scales down afterward.
- Pods access AWS only through a scoped workload role, not node credentials.
- Ingress health checks and Kubernetes probes agree on workload health.

PROJECT 08 / EKS

- The entire cluster and supporting resources can be reproduced through code.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Delete a pod, drain a node and generate load. Demonstrate rescheduling, disruption-budget behavior, scaling and the alarms produced by the event.

Portfolio evidence

- Cluster and workload architecture diagram.
- kubectl output showing rollout, HPA and pod distribution.
- ALB target health and application response.
- Failure-test timeline and recovery observations.

Security and cost guardrails

- EKS has a recurring control-plane cost plus nodes and load balancers; destroy lab clusters promptly.
- Do not expose the Kubernetes API publicly without narrow CIDR restrictions.
- Use workload identity and least privilege; do not give application pods the node role.
- Set requests and limits before testing HPA, or scaling behavior will be misleading.

GitOps and Infrastructure Delivery

Separate infrastructure delivery from application reconciliation: GitHub Actions uses OIDC to plan and apply Terraform, while Argo CD or Flux continuously reconciles Kubernetes desired state from Git.

ARCHITECTURE FLOW

Pull request -> GitHub Actions + OIDC -> Terraform -> AWS -> GitOps repository -> Argo CD -> EKS reconciliation

Primary outcome	Git becomes the audited source of truth for both infrastructure changes and Kubernetes application state, with controlled promotion and drift visibility.
Prerequisites	Projects 2 and 8, GitHub Actions, IAM OIDC federation, Helm or Kustomize and Kubernetes operations.
Key services	GitHub Actions, IAM OIDC, Terraform, EKS, Argo CD / Flux, CloudTrail
Repository	09-gitops-aws-platform plus a separate environment-config repository

Implementation phases

Infrastructure CI

On pull requests, run Terraform format, validate, lint, security checks and plan. Store plan evidence and require review.

Secure apply

Use GitHub OIDC to assume short-lived AWS roles. Apply only from protected environments with approval; never store long-lived AWS keys.

GitOps controller

Install Argo CD or Flux on EKS and point it at an environment repository containing Helm values or Kustomize overlays.

Promotion

Promote immutable image tags through dev, staging and production by pull request. The controller pulls and reconciles the approved state.

Drift and recovery

Demonstrate self-healing after a manual Kubernetes change, controlled rollback through Git revert and clear handling of infrastructure drift.

Required deliverables

- GitHub Actions workflows for Terraform plan, approval and apply using OIDC.
- Restricted OIDC trust policy tied to the repository, branch and environment.
- Argo CD or Flux installation and application definitions.
- Separate application-source and environment-config responsibilities.
- Promotion, rollback, drift and emergency-change runbooks.

Definition of success

- No long-lived AWS credentials exist in GitHub secrets.

PROJECT 09 / GITOPS

- Production Terraform apply requires an approved protected environment.
- A Git change updates the cluster and reports synchronization health.
- A manual Kubernetes drift is detected and either reverted or explicitly accepted.
- A Git revert restores the previous application version.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Manually change a managed Deployment in the cluster. Show the controller detecting and correcting drift, then perform a bad Git promotion and recover through a reviewed revert.

Portfolio evidence

- Pull-request plan and protected-environment approval.
- OIDC role session visible in CloudTrail.
- Argo CD or Flux synchronization and health history.
- Drift correction and Git-based rollback timeline.

Security and cost guardrails

- GitHub Actions plus Terraform is infrastructure CI/CD; the pull-based controller provides the true GitOps reconciliation loop.
- Restrict OIDC trust by repository, branch or environment claims.
- Never allow unreviewed pull requests to apply production infrastructure.
- Protect the environment repository because changing it changes production behavior.

Complete Enterprise Cloud Platform

Combine the strongest patterns from all prior projects into one coherent, multi-environment platform that can deploy, scale, observe, secure and recover a realistic application.

ARCHITECTURE FLOW

GitHub -> CI + IaC -> VPC / ALB -> ECS services -> RDS + S3 -> Observability + security + DR

Primary outcome	A defensible production-style platform with architecture decisions, automated delivery, operational evidence and measured resilience.
Prerequisites	Projects 1 through 9, plus a small WellSpring backend API and PostgreSQL schema using synthetic demonstration data only.
Key services	VPC, ALB, ECS Fargate, RDS, S3, Lambda, CloudWatch, SNS, IAM, Auto Scaling, AWS Backup
Repository	10-enterprise-cloud-platform

Implementation phases

Platform foundation

Create multi-environment network, identity, state, DNS, certificates, encryption, tagging and account-level guardrails through Terraform.

Application architecture

Run frontend and backend containers on ECS Fargate behind ALB, store application data in private RDS and objects in S3, and keep secrets outside images.

Delivery and scaling

Build immutable artifacts, promote through environments, use ECS-native progressive deployment and configure service and database capacity deliberately.

Operations and security

Integrate dashboards, alarms, tracing, centralized findings, image and IaC gates, runbooks and incident notifications.

Reliability and evidence

Automate backups and recovery tests, perform load, security, deployment and disaster game days, then complete a Well-Architected review.

Required deliverables

- Complete Terraform platform with separate environment state and reusable modules.
- Application CI/CD and progressive deployment with health-based rollback.
- Multi-AZ architecture, RDS backups, S3 protection and recovery procedure.
- Dashboards, SLOs, alarms, security findings and operational runbooks.
- Executive architecture document, threat model, cost model and Well-Architected assessment.

Definition of success

- A new environment can be created and an application release deployed without manual server configuration.
- The platform handles a measured load target and scales within defined limits.
- A bad release rolls back automatically while the stable version remains available.

PROJECT 10 / CAPSTONE

- A security or availability incident is detected, notified and handled through a tested runbook.
- A recovery test restores service and data within measured RPO/RTO targets.

Failure test

PROVE RECOVERY, NOT ONLY DEPLOYMENT

Run a full game day: deploy a faulty backend version, terminate tasks, generate load and restore synthetic data. Capture detection, rollback, scaling and recovery evidence in one incident report.

Portfolio evidence

- Executive and detailed architecture diagrams with trust and network boundaries.
- Successful pipeline, deployment, scaling, rollback and recovery timelines.
- Well-Architected findings and prioritized improvement backlog.
- Short demonstration video and a recruiter-friendly project summary.

Security and cost guardrails

- Do not claim the portfolio healthcare application is HIPAA compliant; use synthetic data only.
- Use multiple Availability Zones, private data stores, encryption and temporary credentials.
- Set an AWS Budget, tag all resources and provide automated teardown for demonstration environments.
- Enterprise quality means measured operations and recovery, not merely a long list of services.

Execution rhythm for every project

- 1. Define the user or operational problem and a measurable success condition.
- 2. Draw the initial architecture and identify network, trust and data boundaries.
- 3. Estimate cost, create budget alerts and decide how resources will be destroyed.
- 4. Build the smallest working version manually only when learning a service for the first time.
- 5. Codify the final infrastructure and configuration; avoid undocumented console dependencies.
- 6. Add pipeline quality gates, immutable releases and short-lived credentials.
- 7. Add observability before declaring deployment complete.
- 8. Break the system intentionally and prove rollback or recovery.
- 9. Capture sanitized evidence and write the incident or learning note.
- 10. Destroy temporary infrastructure and verify that no costly orphaned resources remain.

Review scorecard

Architecture	Can a reviewer understand components, boundaries, dependencies and traffic flow in under two minutes?
Automation	Can the environment and release be reproduced without hidden manual steps?
Security	Are credentials temporary, IAM scoped, secrets protected and findings handled safely?
Reliability	Are health, rollback, backup and recovery behaviors tested rather than assumed?
Operations	Are logs, metrics, alarms, ownership and runbooks useful during a real failure?
Cost	Are expensive resources identified, tagged, monitored and automatically cleaned up?
Evidence	Does the repository prove the claims with diagrams, tests, logs and concise explanations?

AWS account safety before Project 1

- Protect the root user with MFA and do not use it for daily work.
- Use IAM Identity Center or an administrative role for human access.
- Create an AWS Budget and billing alerts before launching persistent services.
- Enable CloudTrail and review account contact and recovery information.
- Choose one primary learning region and use it consistently unless testing disaster recovery.
- Adopt tags: Project, Environment, Owner, ManagedBy and ExpiresAt.
- Use synthetic demonstration data only, especially for the WellSpring healthcare-themed application.
- Create a written teardown checklist for ALB, NAT Gateway, RDS, ECS, EKS and cross-region resources.

PORTFOLIO PRINCIPLE

Enterprise quality is not the number of AWS icons in a diagram. It is the ability to explain trade-offs, operate the system during failure and show evidence that recovery works.

Suggested architecture artifacts

- Context diagram: users, external systems and the platform boundary.
- Container or service diagram: application components and service dependencies.
- Deployment diagram: AWS accounts, regions, VPCs, subnets, load balancers and compute.
- Sequence diagram: build, deployment, event processing, rollback or recovery timeline.
- Threat model: trust boundaries, sensitive assets, likely threats and mitigations.

Reference baseline

These sources anchor the roadmap's modern design choices. Service documentation should be rechecked when each project starts because AWS and tooling continue to evolve.

AWS Well-Architected Framework

<https://docs.aws.amazon.com/wellarchitected/latest/framework/definitions.html>

Amazon ECS blue/green deployments

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/deployment-type-blue-green.html>

Amazon ECS canary deployments

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/canary-deployment.html>

AWS guidance: ECS-native versus CodeDeploy

<https://aws.amazon.com/blogs/devops/choosing-between-amazon-ecs-blue-green-native-or-aws-codedeploy-in-aws-cdk/>

Terraform on AWS backend best practices

<https://docs.aws.amazon.com/prescriptive-guidance/latest/terraform-aws-provider-best-practices/backend.html>

Terraform S3 backend and native lockfile

<https://developer.hashicorp.com/terraform/language/backend/s3>

AWS IAM OIDC federation

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers_oidc.html

Jenkins Pipeline and Jenkinsfile

<https://www.jenkins.io/doc/book/pipeline/jenkinsfile/>

Amazon ECR image lifecycle

<https://docs.aws.amazon.com/AmazonECR/latest/userguide/getting-started-cli.html>

AWS Systems Manager Run Command

<https://docs.aws.amazon.com/systems-manager/latest/userguide/run-command.html>

Final note

THE MISSION

Complete the projects one at a time. Prefer a small system that is reproducible, secure and tested under failure over a large system assembled without operational proof.